

Track 5 — Leistungsnachweis-Kette gehärtet

Datum: 28.08.2026

Bereich: Erfassung → OCR/Prüfzentrale → Unterschrift → Leistungsnachweis → Kassenabrechnung (§ 105 / § 302 / DTA)

Ausgangsstand: f45537f — 7532 vitest, 2476 node:test

Live-Nachweis: scripts/verify-leistungsnachweis-live.mjs — 11/11 grün

Migration nötig: nein — jeder Befund liegt im Code, die Datenbank sagt bereits das Richtige

Kurzfassung

Drei Befundklassen, alle drei gegen die Produktionsdatenbank belegt:

1. P0 — Storno-Blindheit in den amtlichen Nachweis- und Exportwegen.

Ein widerrufener Leistungsnachweis stand weiterhin auf dem PDF, das der Pflegekasse vorgelegt wird, und in der EDIFACT-Datei, mit der die Forderung tatsächlich übermittelt wird. Track 1 hatte den Riegel in der Rechnungs-RPC gesetzt (v10, Migration 20261013000000) — sieben weitere Wege lesen ihre Leistungen aber selbst aus service_records und liefern daran vorbei.

2. P0 — Mandanten-Streuung in die Stamm-Organisation.

Drei Dienstschlüssel-Inserts der Prüfkette schrieben organization_id nicht mit. Die Spalte ist NOT NULL mit Default current_org_id(), und diese Funktion liest auth.uid() — beim Dienstschlüssel gibt es keinen angemeldeten Nutzer, die Fallback-Kette endet in einer fest verdrahteten Stamm-Organisation. Jeder Prüffoto-Eintrag, jedes Standort-Ereignis und jeder Geo-Prüfeintrag jedes Mandanten wäre im Bestand der Stamm-Organisation gelandet.

3. P1 — Der Nachweisstand steht in zwei Spalten, der Sync läuft in eine Richtung.

Live tragen 28 von 30 Nachweisen proof_status='ENTWURF', **15 davon bei status='invoiced'** — also längst abgerechnet. Die Automatisierungsketten lasen allein proof_status und legten daraus täglich Aufgaben an: 56 Aufgaben an Betreuungskräfte und PDL für Arbeit, die erledigt und bezahlt ist. Die DTA-Vorprüfung meldete praktisch jeden Nachweis als „nicht unterschrieben“; die SGB-V-Vollständigkeitsprüfung jeden als „nicht abgeschlossen“.

Ergebnis: 84 neue Tests (davon 8 Gegenproben, die die alte Regel noch einmal ausführen), vitest 7532 → 7616, node:test 2476 unverändert, Typecheck 0, lint:forbidden 0 (24768 Dateien).

Block 1 — Storno-Blindheit (P0)

Die Ursache, in einem Satz

'STORNIERT' hat kein Gegenstück im status-Werteset. Der Live-CHECK service_records_status_check kennt nur draft|incomplete|complete|signed|invoiced (im Live-Nachweis als STORNO-2 gelesen). Der Storno-Weg schreibt deshalb nur proof_status/billing_status und lässt status unverändert stehen — ein widerrufener Nachweis bleibt auf signed oder complete. Jede Abfrage der Form

```
.in('status', ['complete', 'signed', 'invoiced'])
```

nimmt ihn damit mit, und zwar unauffällig: die Abfrage liefert weiter Zeilen, nur eben eine zu viel.

Was daran nicht bloß unsauber war

Weg	Was dort entsteht
app/api/leistungsnachweis/route.ts	Das Leistungsnachweis-PDF, das der Pflegekasse zur Anerkennung vorgelegt wird
lib/abrechnung/leistungsnachweis-pdf.ts	Dasselbe Blatt je Verordnung, inkl. Genehmigungsnummer
lib/abrechnung/kassenabrechnung-engine.ts	Die EDIFACT-Datei (§ 105) — der Weg, auf dem die Forderung die Kasse tatsächlich erreicht
lib/abrechnung/sgb-v/versand.ts	Der echte § 302-Lauf an die Datenannahmestelle
app/api/billing/sgb-v/vorschau/route.ts	Die Fallzahl und Summe, die der Anwender als abrechenbar liest
app/api/billing/dta/dry-run/route.ts	Der Trockenlauf, der die Datei vor dem Versand prüft
app/api/billing/auto-invoice/route.ts	Die automatische Rechnungserstellung
app/api/billing/monthly-closing/route.ts	Die Kennzahlen und die Ampel des Monatsabschlusses
lib/analytics/pruefmappe.ts	Die Dokumentationsquote, die bei einer MD-Prüfung vorgelegt wird

Eine widerrufen Leistung auf einem Kassennachweis ist keine Formalie: es ist eine Forderung für etwas, das ausdrücklich zurückgenommen wurde.

Zwei Sonderfälle in auto-invoice

Dieselbe Weglassung hatte dort eine zweite, gegenläufige Wirkung:

1. Ein auf complete storniertes Blatt zählte in der Vollständigkeitsprüfung als „noch nicht unterschrieben“. Der Monat war damit ****dauerhaft nicht abrechenbar**** — und nachträglich unterschreiben geht nicht, der Nachweis ist ja widerrufen. Ein einziges Storno legte die automatische Rechnung für den ganzen Monat still.
2. Ein auf signed storniertes Blatt wanderte in signedIds und wurde im Zweig „alle bereits einer Rechnung zugeordnet“ auf status='invoiced' gestempelt. Die Oberfläche führte den Widerruf danach als abgerechnet.

Abhilfe

ohneStornierte() / istStorniert() aus lib/leistungsnachweis/status-sync.ts — die eine Stelle, an der „nicht abrechenbar“ definiert ist, und dieselbe Rechnung wie das COALESCE in der RPC v10. Wichtig und leicht zu übersehen: ohne proof_status und billing_status im select() sind beide Felder undefined und der Filter entfernt nichts — er sähe geprüft aus und wäre wirkungslos. Beide Spalten sind deshalb überall ergänzt, und der Zaun in __tests__/leistungsnachweis/storno-geldwege-zaun.test.ts prüft beides.

Bewusst NICHT geändert

lib/pdf/rechnung-paket.ts liest seine Nachweise über invoice_items — also genau die Menge, die die RPC v10 bereits gefiltert hat. Ein Storno nach der Abrechnung fängt assertStornierbar() ab. Ein zweiter Filter an dieser Stelle wäre eine Wiederholung ohne Wirkung.

Block 2 — Mandanten-Streuung in die Stamm-Organisation (P0)

Der Mechanismus

ocr_results, review_errors und geo_events tragen organization_id als NOT NULL mit Default current_org_id() (Live-Nachweis MANDANT-2). current_org_id() liest der Reihe nach: app_metadata.org_id aus dem JWT → organization_members → caregivers → clients → und zuletzt eine **fest verdrahtete Stamm-Organisation (MANDANT-1**).

Beim Dienstschlüssel ist auth.uid() NULL. Die ganze Kette greift ins Leere und der Default liefert die Stamm-Organisation — gleich, welcher Mandant den Vorgang ausgelöst hat.

Die drei Stellen

Datei	Insert	Folge
app/api/native/leistungsnachweis-upload/route.ts	ocr_results	Jedes Prüffoto jedes Mandanten landet in der Stamm-Organisation
app/api/native/geo-events/route.ts	geo_events und review_errors	Standort-Ereignisse und Geo-Prüfeinträge ebenso
app/api/admin/ocr/route.ts	ocr_results, review_errors	dito (dort zusätzlich der RLS-Befund aus Block 3)

Die Wirkung geht in beide Richtungen: die eigene Prüfzentrale des Mandanten sieht nichts (der RESTRICTIVE org_fence filtert auf die eigene Organisation), und die Stamm-Organisation sieht die Einträge fremder Mandanten — mit service_record_id, Beschreibung und Entfernungsangabe.

Die Upload-Route kannte auth.organizationId sogar und hatte den Nachweis bereits dagegen gefenced — nur der Insert daneben setzte sie nicht.

Abhilfe

Alle drei Inserts setzen organization_id jetzt ausdrücklich aus dem Auth-Kontext. geo-events bekommt zusätzlich den fehlenden Mandanten-Fence auf den gelesenen service_records-Datensatz.

Restposten (benannt, nicht behoben): der Default current_org_id() liegt live auf über 60 Tabellen. Jeder Dienstschlüssel-Insert im Repo, der organization_id weglässt, hat dieselbe Wirkung. Dieser Track hat die drei Stellen der Leistungsnachweis-Kette geschlossen; ein Repo-weiter Durchgang ist ein eigener Track wert.

Block 3 — Prüfzentrale schrieb mit dem falschen Client, fail-open (P0/P1)

app/api/admin/ocr/route.ts schrieb ocr_results und review_errors mit dem RLS-Client des Aufrufers. Auf beiden Tabellen steht als einzige schreibende Policy *_admin_all mit is_admin(), und is_admin() ist live auf admin|superadmin beschränkt (PRUEFZENTRALE-1/2). Herein lässt die Route über einsatz.schreiben aber auch die PDL — also genau die Rolle, die in

einem Pflegedienst die Prüfzentrale bedient. Für sie scheiterte der Insert an 42501; die Route meldete HTTP 500.

Dieselbe Klasse wie d707cda (QM/PDL-Dashboards) und 48d6f3b (Angehörigenportal).

Schwerer wiegt der zweite Teil: der review_errors-Insert war fail-open. Schlag er fehl, wurde er nur geloggt, und die Route antwortete **200 mit review_errors: []. Die Oberfläche meldete dem Büro eine bestandene Prüfung, während die Beanstandungen nirgends ankamen — darunter signature_missing mit severity: 'critical', also der Hinweis, dass für einen Einsatz **überhaupt keine Unterschrift** vorliegt. Ohne Unterschrift ist der Einsatz nicht abrechenbar; genau das hätte der Eintrag sagen sollen.

Live: ocr_results und review_errors sind beide leer (PRUEFZENTRALE-3) — was zu einem Weg passt, der noch nie erfolgreich geschrieben hat, und einen Backfill entbehrlich macht.

Abhilfe

Dienstschlüssel mit ausdrücklicher organization_id, und fail-closed: schlägt der Befund-Eintrag fehl, wird der ocr_results-Datensatz wieder entfernt (kein Prüfungsvorgang ohne seine Befunde) und die Route antwortet **503 mit Klartext. Bewusst keine** neue RLS-Policy: die Datenbank sagt bereits das Richtige, zu ändern war der Client, der ihr vorauslief.

Block 4 — Zwei Statusspalten, eine Sync-Richtung (P1)

Der Befund

Der Trigger sync_service_record_status bildet proof_status → status ab und schreibt ausschließlich NEW.status (live aus pg_proc gelesen, SYNC-1). Den Rückweg gibt es nirgends — weder als Trigger noch im Code. Jeder Schreibweg, der nur status setzt (die Rechnungs-RPC setzt 'invoiced', der Verwaltungsweg 'signed'), lässt proof_status auf dem alten Wert stehen.

Live gemessen:

- 28 Nachweise mit proof_status='ENTWURF' bei status IN ('signed','invoiced') (DRIFT-1)
- 15 Nachweise mit billing_status='OFFEN' bei status='invoiced' (DRIFT-2)

Bei 30 Nachweisen insgesamt ist proof_status damit für 93 % des Bestands eine falsche Auskunft.

Was daran hing

Stelle	Verhalten vor der Härtung
lib/automation/nachweis-fehlt.ts	Legte für jeden der 28 Nachweise täglich je eine Aufgabe an die Betreuungskraft und an die PDL an — 56 Aufgaben für Arbeit, die längst abgerechnet ist
lib/automation/unterschrift-erinnerung.ts	Erinnerte an Unterschriften zu Einsätzen, die bereits auf einer Rechnung stehen — und wegen is_locked gar nicht mehr unterschrieben werden können
app/api/billing/dta/dry-run/route.ts	Meldete praktisch jeden Nachweis als „nicht unterschrieben“ und begrub damit die vier, bei denen tatsächlich kein Beleg vorliegt

Die neue Regel

Neu in lib/leistungsnachweis/status-sync.ts:

- `nachweisRang(rec)` — Rang auf der status-Skala (0...4), gebildet aus dem höheren der beiden Stände. Unbekannte Werte zählen als –1 und können den Rang damit nur nicht anheben; sie senken ihn nie.
- `nachweisOffen(rec)` — „ist dieser Nachweis noch offen?“ Das ist die Frage der Erinnerungsketten. Storniert zählt als entschieden, nicht als offen.
- `hatUnterschrift(rec)` — „liegt eine Unterschrift vor?“ Diese Frage wird bewusst anders beantwortet: nicht über status, sondern über einen Beleg (`proof_status` UNTERSCHRIEBEN/ABGERECHNET, `signature_hash` oder `client_signature`).

Der Unterschied zwischen den letzten beiden ist der Kern: `status='signed'` kann aus einem direkten Verwaltungsschreibvorgang stammen, ohne dass je jemand unterschrieben hätte. Würde `hatUnterschrift()` den status mitzählen, verschwände genau der Fall, für den die DTA-Vorprüfung da ist. Live sind das 4 von 30 Nachweisen — vorher gingen sie im Rauschen der anderen 26 unter, die alle fälschlich als unsigniert gemeldet wurden.

Warum kein Rückweg-Trigger

Naheliegender wäre gewesen, den Trigger um `status` → `proof_status` zu ergänzen. Das wäre falsch: `status='signed'` würde dann auf `proof_status='UNTERSCHRIEBEN'` abgebildet, und dieser Wert löst über `compute_signature_hash` einen Signatur-Hash aus. Für einen Nachweis ohne Unterschrift hieße das, eine Unterschrift zu behaupten, die es nie gab — schlimmer als die Drift. Die Auskunft gehört auf der Leseseite korrigiert, nicht durch nachträgliches Erfinden von Belegen.

Geänderte Dateien

Neue Regel

- `lib/leistungsnachweis/status-sync.ts` — `nachweisRang`, `nachweisOffen`, `hatUnterschrift`

Storno-Filter ergänzt (Spalten + ohneStornierte)

- `app/api/leistungsnachweis/route.ts`
- `lib/abrechnung/leistungsnachweis-pdf.ts`
- `lib/abrechnung/kassenabrechnung-engine.ts`
- `lib/abrechnung/sgb-v/versand.ts`
- `lib/abrechnung/sgb-v/positionen.ts` (Typ `HkpLeistung` um die Storno-Felder erweitert)
- `lib/abrechnung/sgb-v/leistungsnachweis-service.ts`
- `app/api/billing/sgb-v/vorschau/route.ts`
- `app/api/billing/dta/dry-run/route.ts`
- `app/api/billing/auto-invoice/route.ts`
- `app/api/billing/monthly-closing/route.ts`
- `lib/analytics/pruefmappe.ts`

Nachweisstand aus beiden Spalten

- `lib/automation/nachweis-fehlt.ts`
- `lib/automation/unterschrift-erinnerung.ts`

Mandant / Dienstschlüssel / fail-closed

- app/api/admin/ocr/route.ts
- app/api/native/leistungsnachweis-upload/route.ts
- app/api/native/geo-events/route.ts

Tests (neu)

- __tests__/leistungsnachweis/nachweisstand.test.ts
- __tests__/leistungsnachweis/storno-und-nachweisstand-ketten.test.ts
- __tests__/leistungsnachweis/storno-geldwege-zaun.test.ts
- __tests__/leistungsnachweis/ocr-pruefzentrale-route.test.ts
- __tests__/leistungsnachweis/auto-invoice-storno.test.ts

Live-Nachweis

- scripts/verify-leistungsnachweis-live.mjs

Prüfläufe

Lauf	Ergebnis
scripts/verify-leistungsnachweis-live.mjs	11/11 grün
vitest	7616 bestanden, 38 übersprungen (vorher 7532) — 337 Dateien
node:test	2476 bestanden, unverändert
tsc --noEmit	0 Fehler
npm run lint:forbidden	0 Treffer, 24768 Dateien (FULL)

Offene Punkte für den nächsten Track

1. current_org_id()-Default repo-weit. Über 60 Tabellen tragen ihn. Jeder Dienstschlüssel-Insert ohne organization_id legt seine Zeile in der Stamm-Organisation ab. Ein systematischer Durchgang (Lint-Regel oder Quelltext-Zaun über alle admin.from(...).insert(...)) ist überfällig.
2. CAMT/Banking — weiterhin nur in der Tabellenlage erhoben, nicht gehärtet.
3. ~100 Inline-profiles.role-Lesungen ohne die app_metadata-Einschränkung aus Track 4. app/api/leistungsnachweis/route.ts und app/api/billing/monthly-closing/route.ts gehören dazu.
4. Automationsketten 1-8: der Cron filtert organizations nicht auf aktive Mandanten.